

Smart Inventory and Restocking Assistant

A multi-agent decision system with a chatbot that lives on GitHub Actions

System report · July 2026

~20 s

live reply time, issue to answer

0

API keys and servers required

18 / 18

tests passing in CI

7

agents behind every answer

This report documents the system end to end: what the multi-agent pipeline does, how the GitHub Actions chatbot turns the repository into a working assistant with zero infrastructure, the results of all five demo scenarios, and the engineering work that took the codebase to a clean, tested, production-shaped state. Everything shown here ran live on the repository, including the chatbot replies.

Contents

- 1 What this system is
- 2 Architecture and the seven agents
- 3 The chatbot on GitHub Actions
- 4 Live proof: the repository answers
- 5 Scenario results
- 6 Data model
- 7 Engineering quality and cleanup
- 8 Running and extending it

1 · What this system is

Retail restocking is a coordination problem. When shelf stock runs low, someone has to reconcile five different views of the world: current inventory, recent sales velocity, distribution-center stock, supplier quotes and freight options. This project automates that loop with seven cooperating agents and returns one explainable action: transfer from a warehouse, procure from a supplier, split between both, or simply keep monitoring.

Two properties make the design distinctive:

Deterministic agents. Every agent is rule based: thresholds, moving averages, weighted scoring, distance calculations. The same question always yields the same answer, each step is auditable, and the pipeline runs in milliseconds with no LLM, no API key and no cost.

The repository is the chatbot. Because the brain is free and fast, it can run inside a GitHub Actions job. Opening an issue is the chat interface; the workflow runs the pipeline and posts the recommendation as a comment in about twenty seconds, with no server anywhere.

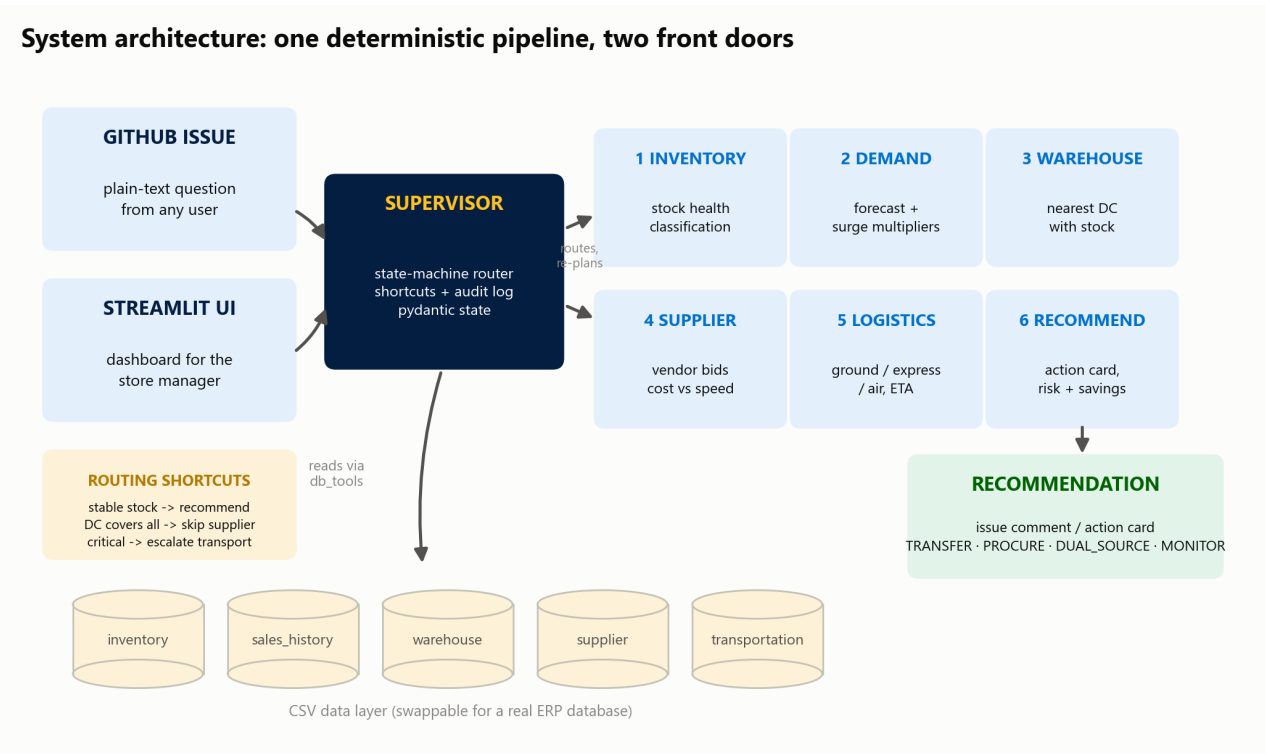


Figure 1 - The pipeline, its two entry points and the data layer

2 · Architecture and the seven agents

A supervisor drives the workflow as a state machine over a typed Pydantic state object. Each agent reads the state, adds its block, and logs what it did; the supervisor then decides the next hop, skipping work that the situation makes irrelevant.

Agent	Decides	Rules it applies
Supervisor	which agent runs next, when to stop	state machine with shortcut routes
Inventory	stock health category	thresholds: critical at 30% of minimum, overstock at 90% of capacity
Demand	7-day forecast and reorder quantity	moving average, promotion and season flags, surge multipliers up to 3.5x
Warehouse	nearest DC that can transfer stock	distance-sorted candidates, transfer cost per mile
Supplier	best vendor when DCs fall short	weighted lead time, reliability and unit cost; speed-weighted under delays

Agent	Decides	Rules it applies
Logistics	ground, express or air, plus ETA	urgency and distance rules; critical stock escalates the mode
Recommendation	the final action card	consolidation with confidence, risk score and cost-avoidance savings

ROUTING SHORTCUTS The shortcuts are what make it feel intelligent: stable stock routes straight to the recommendation, a warehouse that covers the full order skips the supplier entirely, and zero DC stock jumps to vendor sourcing. The audit log records every hop, so a manager can always ask why.

3 · The chatbot on GitHub Actions

The goal was a working chatbot with no infrastructure to babysit. GitHub already hosts the code, the compute and the conversation surface, so the bot uses all three: issues are the chat window, Actions is the runtime, and the built-in GITHUB_TOKEN posts the reply. Nothing to deploy, nothing to pay for, nothing to leak.

The chatbot lives on GitHub Actions: the repository answers its own issues



Measured on the live repository: end-to-end reply in about 20 seconds. No server, no API keys, no cost beyond GitHub's free Actions minutes.

Figure 2 - One question, five steps, about twenty seconds end to end

What you can ask

Ask	Example	What comes back
Product analysis	analyze bottled water at store 1001	full pipeline run and action card
Analysis by IDs	restock Prod_003 at Store_1002	same, exact record match
Store overview	status of store 1004	health table for every product
Demo scenario	run scenario 2	scenario with database overrides, sandboxed
Coverage	list products	known stores and products
Guide	help	usage table

Plain-text parser. Products match by ID or fuzzy name (milk, bottled water, smart tv); stores by number. Unmatchable questions get the help card, never a stack trace.

Sandboxed scenarios. Scenario overrides run against a snapshot and the data files are restored afterwards, verified by a dedicated test.

Loop safe. The workflow ignores bot-authored events and pull requests, and serializes runs per issue, so the bot can never argue with itself.

4 · Live proof: the repository answers

Both triggers were exercised on the live repository immediately after deployment. Issue #1 asked a plain-text question; a follow-up comment asked for a scenario.

Event	Trigger	Workflow result	Time
Issue #1 opened: "analyze bottled water at store 1001"	issues: opened	success, full analysis posted	20 s
Comment: "run scenario 3"	issue_comment: created	success, scenario card posted	~20 s
Push of the chatbot itself	push	CI success: 18 tests + smoke checks	36 s

The actual first reply (excerpt)

```
### Purified Bottled Water 40-Pack at Store_1001
| Stock health | CRITICAL_UNDERSTOCK (0 on hand, threshold 50) |
| Demand | 119 units over 7 days (SPIKE), reorder 400 |
| Warehouse | Denver DC: transfer 400 units |
| Logistics | AIR_CARGO, ETA 0.5 days |
Recommendation: TRANSFER confidence 95/100 · risk 65/100
```

AUDITABLE BY DESIGN The reply includes a collapsible agent execution log: every hop the supervisor took, every number each agent produced, and why the supplier step was skipped. Explainability is part of the interface, not an afterthought.

5 · Scenario results

The five demo scenarios stress different routes through the supervisor. All five were run end to end through the chatbot pipeline; the numbers below are actual outputs, not illustrations.

TRANSFER

All five demo scenarios, run end to end through the chatbot pipeline

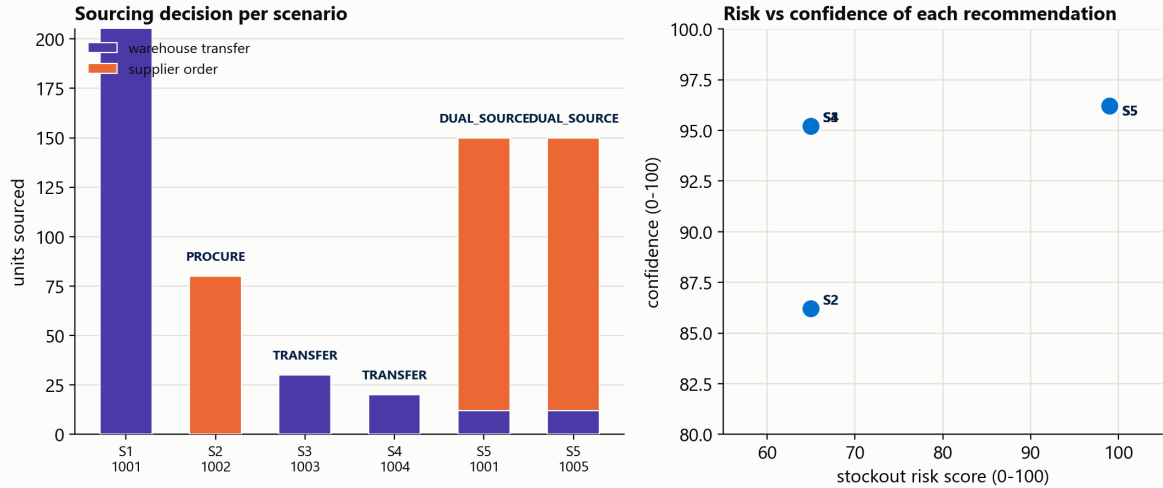


Figure 3 - Sourcing decisions and recommendation quality across the scenario suite

#	Store	Product	Action	Sourcing	Transport	Conf.
S1	store 1001	Purified Bottled Water 40-Pack	TRANSFER	400 u from Denver DC	air cargo	95
S2	store 1002	Equate Allergy Relief Cetirizine	PROCURE	80 u from Nestle Foods	air cargo	86
S3	store 1003	onn. 50-inch 4K UHD Smart TV	TRANSFER	30 u from Miami DC	air cargo	95
S4	store 1004	Mainstays Metal Outdoor Patio Set	TRANSFER	20 u from New York DC	air cargo	95
S5	store 1001	Great Value Organic Whole Milk	DUAL_SOURCE	12 u from Dallas DC + 138 u from Dairy Farms Inc.	express motor	96
S5	store 1005	Great Value Organic Whole Milk	DUAL_SOURCE	12 u from Dallas DC + 138 u from Dairy Farms Inc.	express motor	96

Scenario 5 runs the same product for two stores and produces an independent sourcing plan for each, which is exactly what the contention story needs: the warehouse can only cover 12 units per store, so both plans dual-source the remainder from the dairy supplier.

6 · Data model

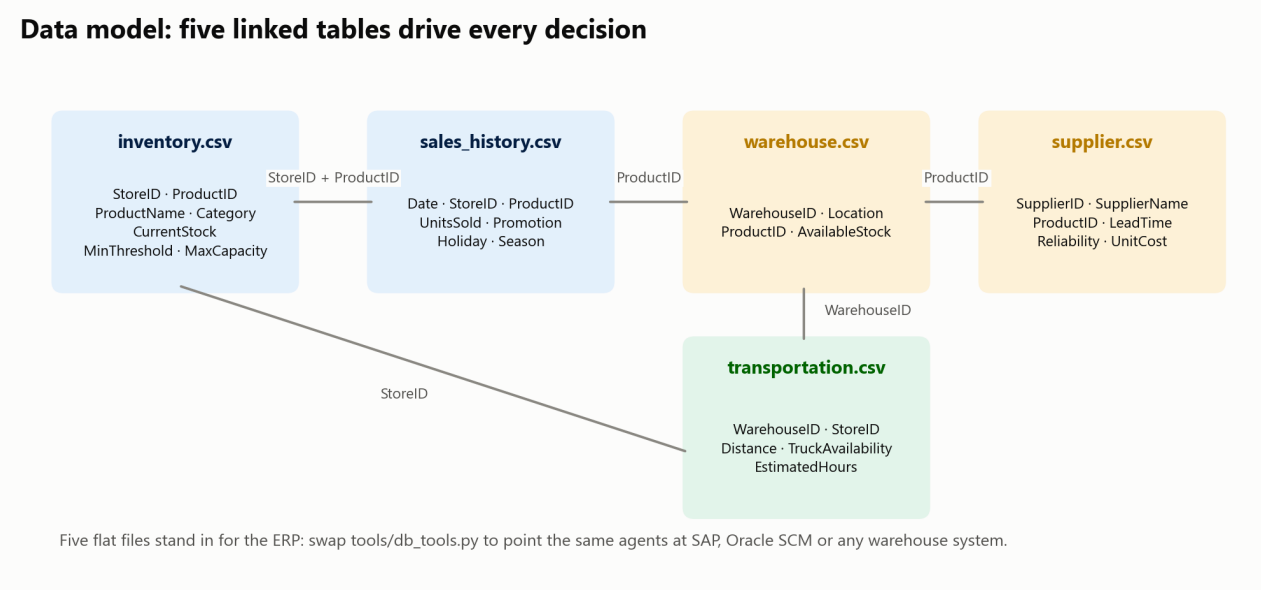


Figure 4 - Five linked tables drive every decision

The synthetic dataset covers 5 stores, 8 products across grocery, pharmacy, electronics and seasonal categories, 8 distribution centers, per-product supplier quotes and 30 days of sales history with promotion and holiday flags. Note for future work: the generator draws store-to-DC distances at random, so some legs are unrealistically long; regenerating with geography-aware distances would make the logistics costs look as sensible as the routing logic is.

7 · Engineering quality and cleanup

Test and automation status

Check	Coverage	Status
Agent unit tests	stock health, forecasting, supplier scoring, logistics rules	PASS (11 tests)
Chatbot tests	parser intents, live answers, scenario sandboxing, catalogs	PASS (7 tests)
CI on every push	full pytest suite plus three chatbot smoke checks	PASS (36 s)
Chatbot workflow	replies to issues and comments, bot-loop guarded	LIVE (2 runs, both success)

Repository cleanup applied

Removed a credential-reading script. `tools/github_helper.py` parsed the developer's `~/.git-credentials` file to call the GitHub API. Unused by the application and unsafe to ship; deleted.

Removed dead configuration. `.env.example` promised `APP_ENV`, `LOG_LEVEL` and weight variables that no code reads; deleted rather than pretend.

README rewritten. Now leads with the chatbot, documents real behavior, and fixes a license link that pointed at a file on the original author's local disk.

Consistent hygiene. No em dashes, no marketing filler, issue template for the assistant, badges wired to the actual workflows.

8 · Running and extending it

```
pip install -r requirements.txt
pytest tests/ -q # 18 tests
streamlit run app.py # dashboard
python -m chatbot.bot "run scenario 4" # chatbot, locally
```

Swap the data layer. `tools/db_tools.py` is the only file that touches storage; point it at a real inventory database and every agent follows.

Upgrade the forecast. The demand agent's moving average sits behind a clean interface; an ML model can replace it without touching the supervisor.

Close the loop. The recommendation agent's action card has everything a purchase-order API needs; wiring it up turns advice into execution.

Repository: github.com/ShreyasThakur11/walmart-agentic-ai · chatbot verified live on issue #1 · report generated from actual pipeline runs